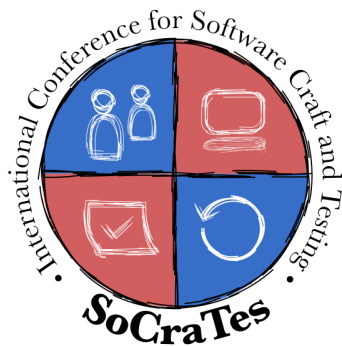


Property-based Testing

Eine Einführung in FsCheck und CsCheck

Patrick Drechsler





Patrick Drechsler

- Software Developer
- Work: C#
- Interests:
 - Software Crafting
 - Test-Driven Development
 - Funktional Programming
 - Domain-Driven Design
- Slides are online: See QR-Code





Roadmap 1/2

- Example-based Testing
- Hello-World FsCheck
- `Property` attribute
- Arbitraries, Generators, and Shrinkers
- Strategies
- CsCheck: an alternative?
- Summary



Roadmap 2/2

PBT framework	Test framework	Language	PBT feature
FsCheck	Xunit	F#	Check.Quick / Check.QuickThrowOnFailure
FsCheck	Xunit	C#	Check.Quick / Check.QuickThrowOnFailure
FsCheck	NUnit	F#	Check.Quick / Check.QuickThrowOnFailure
FsCheck	NUnit	C#	Check.Quick / Check.QuickThrowOnFailure
FsCheck	Xunit	F#	Property attribute (no arguments)
FsCheck	Xunit	F#	Property attribute (with primitive arguments)
FsCheck	Xunit	F#	Properties, Arbitraries and Generators
FsCheck	Xunit	F#	Strategies: Randomness, Invariants, Nuclear, There-And-Back, etc
CsCheck	Xunit	F#	Syntax?



Example based tests

Most testing strategies are "Example based".

Example:

```
public void Add(int a, int b)
    ⇒ a + b;
```

C#

Writing tests covering the corner cases is simple.

```
[Fact]
public void Add_works()
    ⇒ Add(1, 2).Should().Be(3);
```

C#

More example data with parameterized test:

```
[Theory]
[InlineData(1, 2, 3)]
[InlineData(0, 1, 1)]
public void Add_works(int a, int b, int expected)
    ⇒ Add(a, b).Should().Be(expected);
```

C#



PBT Hello World (in .NET)

We start without a testing framework like xUnit/NUnit/MSTest.

Let's start with an interactive F# session (aka ``fsi/fsx``)!

```
let reverseList (aList: int list) : int list =  
    failwith "TODO"
```

F#

```
open Xunit  
open FsCheck          // PBT library  
open Swensen.Unquote // Assertion library  
  
[<Fact>]  
let ``reversing a list twice returns original list`` () =  
    let checkFn (aList: int list) =  
        let actual = aList ▷ reverseList ▷ reverseList  
        let expected = aList  
        test <@ actual = expected @> // Assertion  
  
    Check.Quick checkFn // PBT Magic
```

F#



Playground magic move

```
[<Fact>]
let ``reversing a list twice returns original list`` () =
  let checkFn (aList: int list) =
    test <@ aList > reverseList > reverseList = aList @> // Assertion

  Check.Quick checkFn // PBT Magic
```

To navigate backwards, use arrow-left.



Playground continued

```
open Xunit
open FsCheck
open FsCheck.Xunit

[<Property>]
let ``reversing a list twice returns original list`` (aList: int list) =
    let actual = aList ▷ List.rev ▷ List.rev
    actual = aList
```




Gilded Rose

TODO: Check if this is a good example for this talk...

Use <https://github.com/emilybache/GildedRose-Refactoring-Kata/tree/main/csharp.xUnit> repository



Strategies

- Randomness
- Nuclear Explosion
- There And Back Again
- Test Oracle

Strategy: Randomness

Examples:

- "Pick One"

Strategy: Nuclear Explosion

Test that nothing throws

Strategy: There And Back Again

Examples:

- Serialize/Deserialize

Strategy: Test Oracle

Compare 2 different implementations



TestContainers and PBT: A good idea?

TODO



F# Crash Course

- Syntax: OCaml based
- Immutable by default
- Piping
- Records & Discriminated Unions
- Type Inference
- Currying / Partial Application



Thank You!

-  patrick.drechsler@mathema.de
-  <https://github.com/draptik>
-  <https://draptik.github.io/talks/>
-  <https://floss.social/@drechsler>
-  <https://bsky.app/profile/drechsler.bsky.social>
-  <https://www.linkedin.com/in/patrick-drechsler-draptik/>



Slides 

- QR Code or
- <https://draptik.github.io/2024-11-ddc-mutation-testing/>
- sample code: <https://github.com/draptik/2024-mutation-testing>

Image sources: pixabay.com and perchance.org/ai-photo-generator